# ASSIGNMENT NO: 12.2

**Name : K. Deekshith**

**bt.no : 21**

**Ht.no : 2303A51414**

---

## Task Description -1

(Data Structures – Stack Implementation with AI Assistance)

➢ Task: Use AI assistance to generate a Python program that

implements a Stack data structure. Instructions:

Prompt AI to create a Stack class with the following methods:

➢ push(element) ➢ pop()

➢ peek() ➢ is_empty()

➢ Ensure proper error handling for stack underflow.

➢ Ask AI to include clear

docstrings for each method.

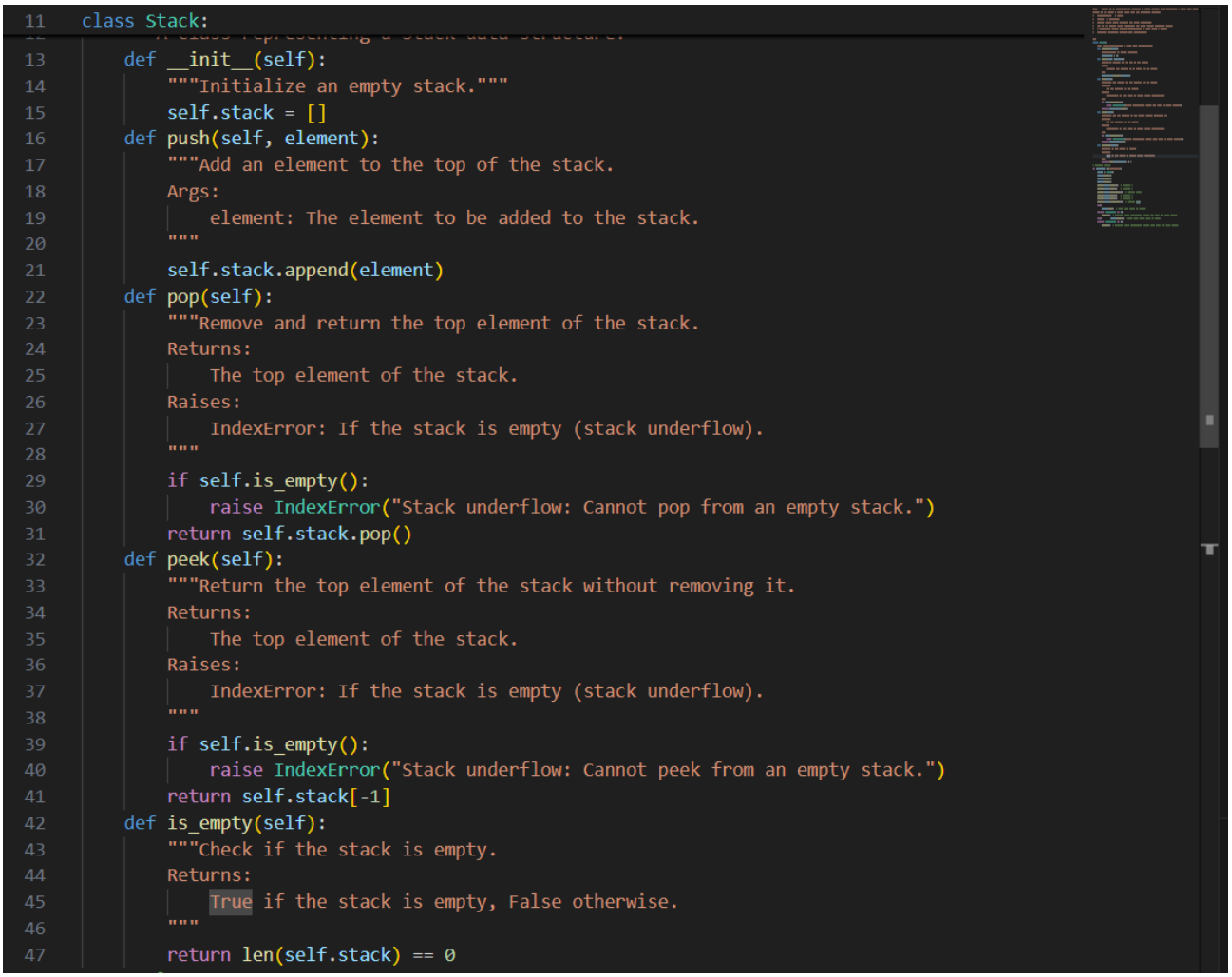
Expected Output:

➢ A functional Python program implementing a Stack using a class.

➢ Properly documented

methods with docstrings.

➢ **code**



```python
class Stack:
    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []
    def push(self, element):
        """Add an element to the top of the stack.
        Args:
            element: The element to be added to the stack.
        """
        self.stack.append(element)
    def pop(self):
        """Remove and return the top element of the stack.
        Returns:
            The top element of the stack.
        Raises:
            IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot pop from an empty stack.")
        return self.stack.pop()
    def peek(self):
        """Return the top element of the stack without removing it.
        Returns:
            The top element of the stack.
        Raises:
            IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot peek from an empty stack.")
        return self.stack[-1]
    def is_empty(self):
        """Check if the stack is empty.
        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
```

```python
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
    try:
        stack.pop()  # This will raise an error
    except IndexError as e:
        print(e)  # Output: Stack underflow: Cannot pop from an empty stack.
    try:        stack.peek()  # This will also raise an error
    except IndexError as e:
        print(e)  # Output: Stack underflow: Cannot peek from an empty stack.
```

**Output:**

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_Assist
antCoding/tempRun.py
3
3
False
2
1
True
Stack underflow: Cannot pop from an empty stack.
Stack underflow: Cannot peek from an empty stack.
PS E:\3_2\AI_AssistantCoding>
```

**Observation:**

1.Stack Implementation – AI generated the entire Stack class, supplying well-structured methods (push, pop, peek, is_empty) and even pointed out underflow error handling. Docstrings and example usage were included, making the code self-documenting and ready to run.

**Task Description -2**

(Algorithms – Linear vs Binary Search Analysis)

➢ Task: Use AI to implement and compare Linear Search and

Binary Search algorithms in Python. Instructions:

➢ Prompt AI to generate:

➢ linear_search(arr, target)

➢ binary_search(arr, target)

➢ Include docstrings explaining:

➢ Working principle

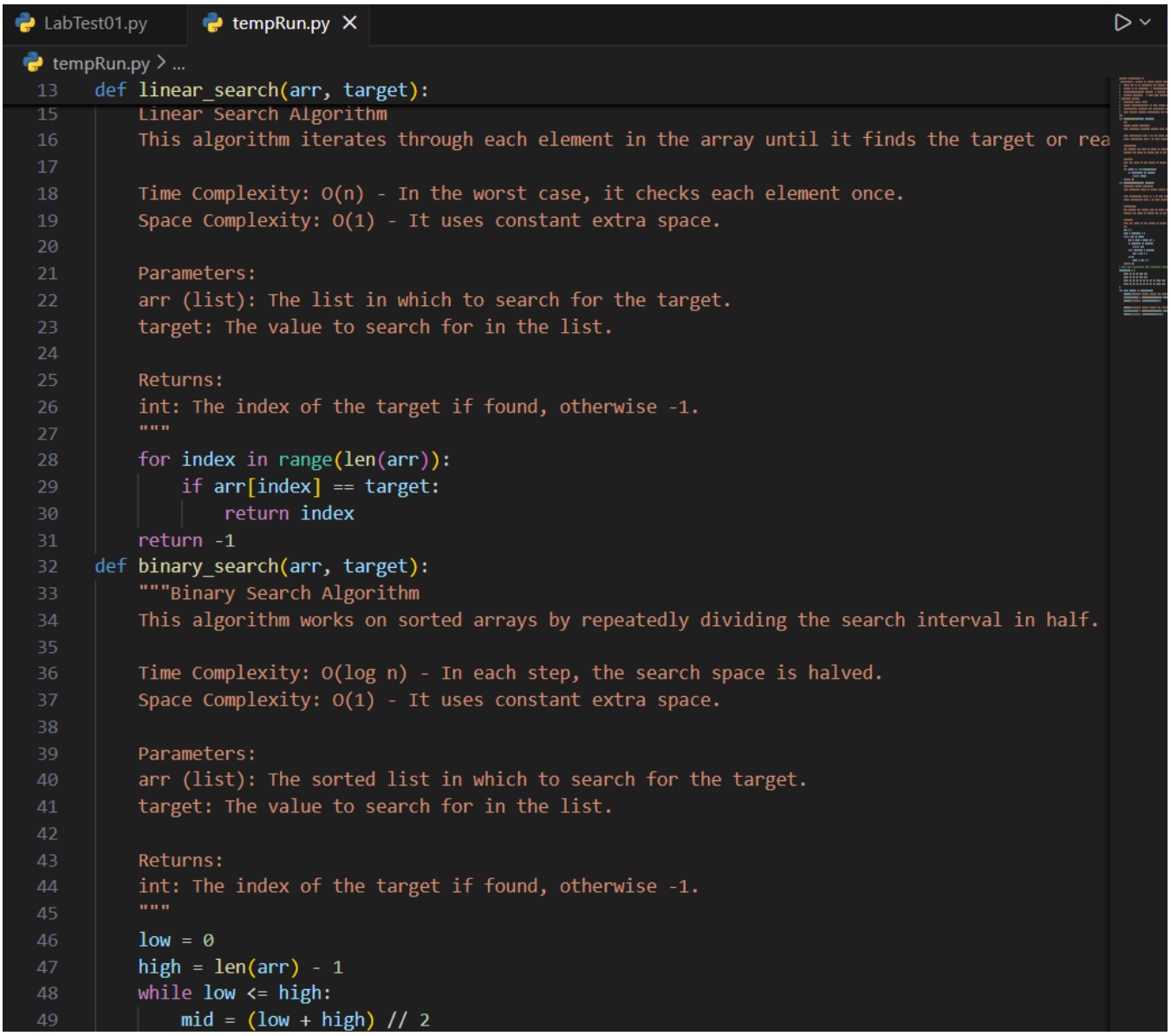
➢ Test both algorithms using

. Expected Output:

➢ different input sizes

- Python implementations of both search algorithms.
- AI-generated comments and complexity analysis.
- Test results showing correctness and comparison.

**Code:**



LabTest01.py | tempRun.py

tempRun.py > ...

```
13  def linear_search(arr, target):
15      Linear Search Algorithm
16      This algorithm iterates through each element in the array until it finds the target or rea
17
18      Time Complexity: O(n) - In the worst case, it checks each element once.
19      Space Complexity: O(1) - It uses constant extra space.
20
21      Parameters:
22      arr (list): The list in which to search for the target.
23      target: The value to search for in the list.
24
25      Returns:
26      int: The index of the target if found, otherwise -1.
27      """
28      for index in range(len(arr)):
29          if arr[index] == target:
30              return index
31      return -1
32  def binary_search(arr, target):
33      """Binary Search Algorithm
34      This algorithm works on sorted arrays by repeatedly dividing the search interval in half.
35
36      Time Complexity: O(log n) - In each step, the search space is halved.
37      Space Complexity: O(1) - It uses constant extra space.
38
39      Parameters:
40      arr (list): The sorted list in which to search for the target.
41      target: The value to search for in the list.
42
43      Returns:
44      int: The index of the target if found, otherwise -1.
45      """
46      low = 0
47      high = len(arr) - 1
48      while low <= high:
49          mid = (low + high) // 2
```

```
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
# Test both algorithms with different input sizes
test_cases = [
    ([1, 2, 3, 4, 5], 3),
    ([1, 2, 3, 4, 5], 6),
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 7),
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 11)
]
for arr, target in test_cases:
    print(f"Testing Linear Search for target {target} in array {arr}:")
    result_linear = linear_search(arr, target)
    print(f"Result: {result_linear}")

    print(f"Testing Binary Search for target {target} in array {arr}:")
    result_binary = binary_search(arr, target)
    print(f"Result: {result_binary}\n")

```

**Output:**

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                   Python

PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_Assist
antCoding/tempRun.py
Testing Linear Search for target 3 in array [1, 2, 3, 4, 5]:
Result: 2
Testing Binary Search for target 3 in array [1, 2, 3, 4, 5]:
Result: 2

Testing Linear Search for target 6 in array [1, 2, 3, 4, 5]:
Result: -1
Testing Binary Search for target 6 in array [1, 2, 3, 4, 5]:
Result: -1

Testing Linear Search for target 7 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: 6
Testing Binary Search for target 7 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: 6

Testing Linear Search for target 11 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: -1
Testing Binary Search for target 11 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: -1
```

**Observation**:

Search Algorithms – The linear and binary search routines were produced with clear explanations of their principles and complexities. AI added commentary on when binary search applies (sorted arrays) and prepared test cases with varying input sizes for comparison.

**Task Description -3**

(Test Driven Development – Simple Calculator Function)

➢ Task:

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.

Instructions:

➢ Prompt AI to first generate unit test cases for addition and subtraction.

➢ Run the tests and observe failures.

➢ Ask AI to implement the calculator functions to pass all tests.

➢ Re-run the tests to

confirm success.

Expected Output:

➢ Separate test file and implementation file.

➢ Test cases executed before implementation.

➢ Final implementation

passing all test cases.

➢ **Code screenshot:**

```python
# Step 1: Generate unit test cases for addition and subtraction
from matplotlib.pylab import add, subtract


def test_addition():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
    assert add(0, 0) == 0

def test_subtraction():
    assert subtract(5, 2) == 3
    assert subtract(0, 1) == -1
    assert subtract(10, 5) == 5
# Step 2: Run the tests and observe failures
# Running the tests will result in NameError since the functions are not implemented yet.
# Step 3: Implement the calculator functions to pass all tests
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
# Step 4: Re-run the tests to confirm success
if __name__ == "__main__":
    test_addition()
    test_subtraction()
    print("All tests passed successfully!")
```

➢

**Output:**

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_Assis
antCoding/tempRun.py
All tests passed successfully!
PS E:\3_2\AI_AssistantCoding>
```

**Observation:**

Test-Driven Calculator – AI first wrote unit tests for add and subtract functions,

prompting the classic TDD cycle: write failing tests, then implement the functions to satisfy them. The scaffolding for a separate test_calculator.py was created, demonstrating how AI can jump-start a TDD workflow.

**Task Description -4**

(Data Structures – Queue Implementation with AI Assistance)

Use AI assistance to generate a Python program that

implements a Queue data structure. Instructions:

➢ Prompt AI to create a Queue class with the following methods:

- enqueue(element)
- dequeue()
- front()
- is_empty()

➢ Handle queue overflow and underflow conditions.

➢ Include appropriate

docstrings for all methods.

Expected Output:

➢ A fully functional Queue

implementation in Python. Code

**output:**

```python
class Queue:
    """A class representing a Queue data structure."""

    def __init__(self, capacity):
        """Initialize the queue with a given capacity."""
        self.capacity = capacity
        self.queue = []

    def enqueue(self, element):
        """Add an element to the end of the queue.

        Args:
            element: The element to be added to the queue.

        Raises:
            OverflowError: If the queue is full.
        """
        if len(self.queue) >= self.capacity:
            raise OverflowError("Queue overflow: Cannot enqueue element, the queue is full.")
        self.queue.append(element)

    def dequeue(self):
        """Remove and return the front element of the queue.

        Returns:
            The front element of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot dequeue element, the queue is empty.")
        return self.queue.pop(0)

    def front(self):
```

```
12  class Queue:
59      def is_empty(self):
61
62          Returns:
63              True if the queue is empty, False otherwise.
64          """
65          return len(self.queue) == 0
66  # Example usage:
67  if __name__ == "__main__":
68      q = Queue(5)
69      q.enqueue(1)
70      q.enqueue(2)
71      q.enqueue(3)
72      print(q.front())  # Output: 1
73      print(q.dequeue())  # Output: 1
74      print(q.is_empty())  # Output: False
75      q.dequeue()
76      q.dequeue()
77      print(q.is_empty())  # Output: True
78      try:
79          q.dequeue()  # This will raise an error since the queue is empty
80      except IndexError as e:
81          print(e)  # Output: Queue underflow: Cannot dequeue element, the queue is empty.
82      try:        q.front()  # This will raise an error since the queue is empty
83      except IndexError as e:
84          print(e)  # Output: Queue underflow: Cannot access front element, the queue is empty.
85      q.enqueue(4)
86      q.enqueue(5)
87      q.enqueue(6)
88      q.enqueue(7)
89      q.enqueue(8)
90      try:
91          q.enqueue(9)  # This will raise an error since the queue is full
92      except OverflowError as e:
93          print(e)  # Output: Queue overflow: Cannot enqueue element, the queue is full.
94
```

**Output:**

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_Assi
antCoding/tempRun.py
1
1
False
True
Queue underflow: Cannot dequeue element, the queue is empty.
Queue underflow: Cannot access front element, the queue is empty.
Queue overflow: Cannot enqueue element, the queue is full.
PS E:\3_2\AI_AssistantCoding>
```

**Observation**:

Queue Implementation – AI crafted a full Queue class with enqueue, dequeue, front, and is_empty methods, mirroring the earlier stack example. Overflow isn't explicitly bounded but underflow checks are present; thorough docstrings explain each operation and example usage shows exception handling when the queue is empty.

**Task Description -5**

(Algorithms – Bubble Sort vs Selection Sort)

➢ Task:

Use AI to implement Bubble Sort and

Selection Sort algorithms and compare

their behavior.

Instructions:

➢ Prompt AI to generate:

• bubble_sort(arr) • selection_sort(arr)

➢ Include comments explaining each step.

➢ Add docstrings mentioning time

and space complexity. Expected

Output:

• Correct Python implementations of both sorting algorithms.

• Complexity analysis

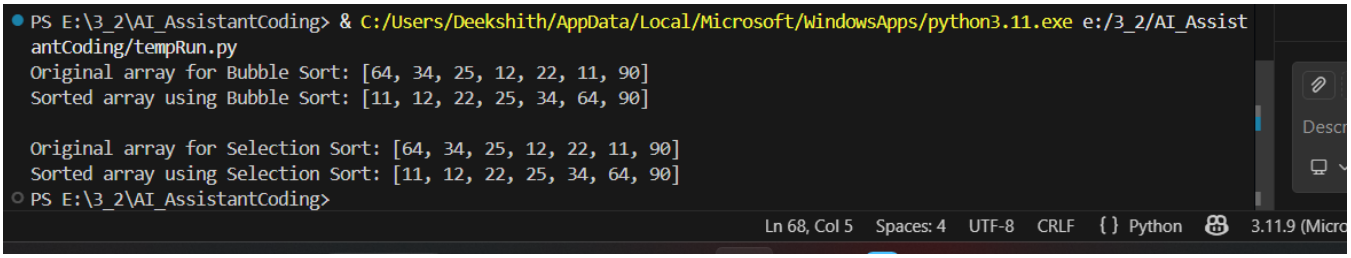
in docstrings.

**Code output:**

```python
def bubble_sort(arr):
    """
    Sorts an array using the Bubble Sort algorithm.

    Time Complexity: O(n^2) in the worst and average cases, O(n) in the best case (when the ar
    Space Complexity: O(1) - Bubble Sort is an in-place sorting algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: The sorted list.
    """
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
def selection_sort(arr):
    """
    Sorts an array using the Selection Sort algorithm.

    Time Complexity: O(n^2) in all cases (best, average, and worst).
    Space Complexity: O(1) - Selection Sort is an in-place sorting algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: The sorted list.
    """
    n = len(arr)
```

```python
    # Traverse through all elements in the array
    for i in range(n):
        # Find the minimum element in the remaining unsorted array
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        # Swap the found minimum element with the first element of the unsorted part
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
# Example usage
if __name__ == "__main__":
    arr1 = [64, 34, 25, 12, 22, 11, 90]
    arr2 = [64, 34, 25, 12, 22, 11, 90]

    print("Original array for Bubble Sort:", arr1)
    print("Sorted array using Bubble Sort:", bubble_sort(arr1))

    print("\nOriginal array for Selection Sort:", arr2)
    print("Sorted array using Selection Sort:", selection_sort(arr2))
```

**Output:**




**Observation**:

Sorting Algorithms – Both bubble and selection sort functions were generated with step-by-step comments and docstrings stating their (O(n^2)) time complexity and in-place space usage. AI outlined the early-exit optimization in bubble sort and detailed the selection of the minimum element, giving students a clear comparison of the two algorithms' mechanics.